Natural Language Processing
Mod - 3: Structures

Theories of parsing

Parsing is the process of examining the grammatical structure and relationships inside a given sentence or text in natural language processing (NLP). It involves analyzing the text to determine the roles of specific words, such as nouns, verbs, and adjectives, as well as their interrelationships.

This analysis produces a structured representation of the text, allowing NLP computers to understand how words in a phrase connect to one another. Parsers expose the structure of a sentence by constructing parse trees or dependency trees that illustrate the hierarchical and syntactic relationships between words.

This essential NLP stage is crucial for a variety of language understanding tasks, which allow machines to extract meaning, provide coherent answers, and execute tasks such as machine translation, sentiment analysis, and information extraction.

Types of parsing in NLP: The types of parsing are the core steps in NLP, allowing machines to perceive the structure and meaning of the text, which is required for a variety of language processing activities. There are two main types of parsing in NLP which are as follows:

Syntactic parsing

Syntactic parsing deals with a sentence's grammatical structure. It involves looking at the sentence to determine parts of speech, sentence boundaries, and word relationships. The two most common approaches included are as follows:

1. **Constituency parsing:** Constituency parsing builds parse trees that break down a sentence into its constituents, such as noun phrases and verb phrases. It displays a sentence's hierarchical structure, demonstrating how words are arranged into bigger grammatical units.
2. **Dependency parsing:** Dependency parsing depicts grammatical links between words by constructing a tree structure in which each word in the sentence is dependent on another. It is frequently used in tasks such as information extraction and machine translation because it focuses on word relationships such as subject-verb-object relations.

Semantic parsing

Semantic parsing goes beyond syntactic structure to extract a sentence's meaning or semantics. It attempts to understand the roles of words in the context of a certain task and how they interact with one another. Semantic parsing is utilized in a variety of NLP applications, such as question answering, knowledge base populating, and text understanding. It is essential for activities requiring the extraction of actionable information from text.

Parsing techniques in NLP

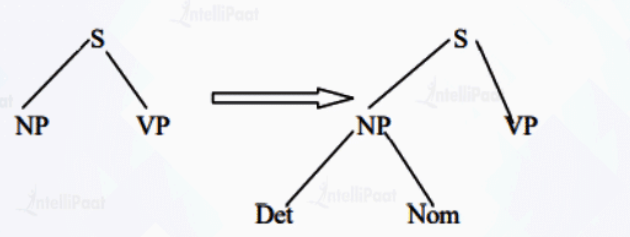
The fundamental link between a sentence and its grammar is derived from a parse tree. A parse tree is a tree that defines how the grammar was utilized to construct the sentence. There are mainly two parsing techniques, commonly known as top-down and bottom-up parsing.

Top-down parsing

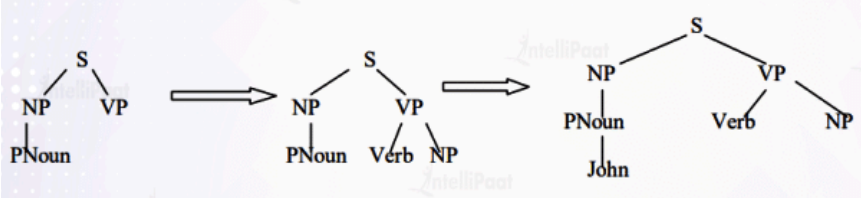
- A parse tree is a tree that defines how the grammar was utilized to construct the sentence. Using the top-down approach, the parser attempts to create a parse tree from the root node S down to the leaves.
- The procedure begins with the assumption that the input can be derived from the selected start symbol S.
- The next step is to find the tops of all the trees that can begin with S by looking at the grammatical rules with S on the left-hand side, which generates all the possible trees.
- Top-down parsing is a search with a specific objective in mind.
- It attempts to replicate the initial creation process by rederiving the sentence from the start symbol, and the production tree is recreated from the top down.
- Top-down, left-to-right, and backtracking are prominent search strategies that are used in this method.

- The search begins with the root node labeled S, i.e., the starting symbol, expands the internal nodes using the next productions with the left-hand side equal to the internal node, and continues until leaves are part of speech (terminals).
- If the leaf nodes, or parts of speech, do not match the input string, we must go back to the most recent node processed and apply it to another production.

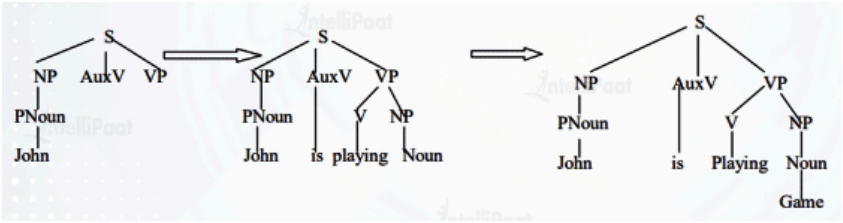
Let's consider the grammar rules:
Sentence = S = Noun Phrase (NP) + Verb Phrase (VP) + Preposition Phrase (PP)
Take the sentence: "John is playing a game", and apply Top-down parsing.



If part of the speech does not match the input string, backtrack to the node NP.



Part of the speech verb does not match the input string, backtrack to the node S, since PNoun is matched.

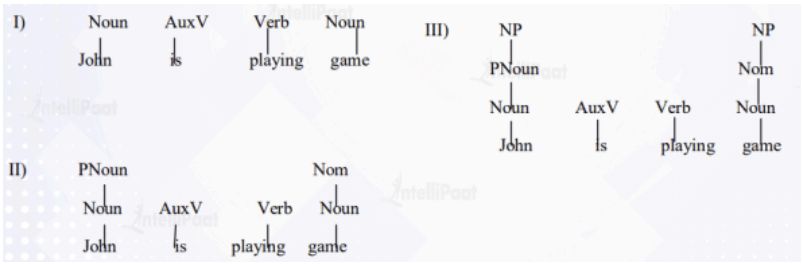


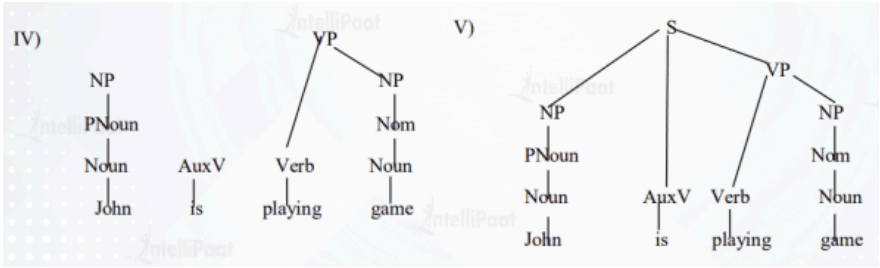
The top-down technique has the advantage of never wasting time investigating trees that cannot result in S, which indicates it never examines subtrees that cannot find a place in some rooted tree.

Bottom-up parsing

- Bottom-up parsing begins with the words of input and attempts to create trees from the words up, again by applying grammar rules one at a time.
- The node is successful if it builds a tree rooted in the start symbol S that includes all of the input. Bottom-up parsing is a type of data-driven search. It attempts to reverse the manufacturing process and return the phrase to the start symbol S.
- It reverses the production to reduce the string of tokens to the beginning Symbol, and the string is recognized by generating the rightmost derivation in reverse.
- The goal of reaching the starting symbol S is accomplished through a series of reductions; when the right-hand side of some rule matches the substring of the input string, the substring is replaced with the left-hand side of the matched production, and the process is repeated until the starting symbol is reached.
- Bottom-up parsing can be thought of as a reduction process. Bottom-up parsing is the construction of a parse tree in postorder.

Considering the grammatical rules stated above and the input sentence "John is playing a game", the bottom-up parsing operates as follows:





How does the parser work?

The first step is to identify the sentence's subject. The parser divides the text sequence into a group of words that are associated with a phrase. So, this collection of words that are related to one another is referred to as the subject. Syntactic parsing and parts of speech are context-free grammar structures that are based on the structure or arrangement of words. It is not determined by the context. The most important thing to remember is that grammar is always syntactically valid, even if it may not make contextual sense.

Aspect	Top-down parsing	Bottom-up parsing
Starting point	Starts from the start symbol and tries to derive the input string, expanding nonterminals into terminals.	Starts from the input tokens and tries to reduce them step by step to the start symbol.
Direction of tree construction	Parse trees are built from root to leaves, matching the way derivations are usually written.	Parse trees are built from leaves to root, as reductions gradually build higher-level constructs.
Derivation used	Implements a leftmost derivation of the input string (from start symbol to terminals).	Implements the rightmost derivation in reverse by repeatedly reducing handles.
Typical implementation	Often implemented as recursive-descent or table-driven LL(k) predictive parsers.	Often implemented as shift–reduce parsers using LR, SLR, LALR, or LR(1) parse tables.
Grammar restrictions	Cannot handle left-recursive grammars directly; grammar must be transformed to remove left recursion and often left-factorized.	Can handle a much broader class of context-free grammars, including many that are left-recursive.
Power / expressiveness	Less powerful: restricted to LL-type grammars that satisfy certain non-ambiguity and lookahead constraints.	More powerful: LR-type parsers can recognize most deterministic context-free grammars used in programming languages.
Ease of construction	Simple to understand and hand-code; good for small or simple languages or educational use.	More complex to construct; parse tables are usually generated by parser generators.
Error reporting	Typically provides more intuitive, localized error messages because it knows which nonterminal is being expanded when failure occurs.	Error messages can be less intuitive, though LR tools add heuristics for better error recovery.
Performance	Fast for simple grammars; overhead mainly from function calls and possibly backtracking if not predictive.	Very efficient and deterministic for large real-world grammars once tables are built, often preferred in production compilers.
Use in practice	Common in educational compilers, or where simplicity and hand-written parsers are desired.	Dominant in industrial-strength compilers and many parser generators

Applications of parsing in NLP

1. **Syntactic analysis**: Parsing helps in determining the syntactic structure of sentences by detecting parts of speech, phrases, and grammatical relationships between words. It is critical for understanding sentence grammar.
2. **Named Entity Recognition (NER):** It parsers can detect and classify entities in text, like people's, organizations, and locations' names, and other things. It is essential for information extraction and text comprehension.
3. **Semantic Role Labeling (SRL):** SRL parsers determine the semantic roles of words in a sentence, such as who is the "agent," "patient," or "instrument" in a given activity. It is essential for understanding the meaning.
4. **Machine translation:** Parsing can be used to assess source language syntax and generate syntactically correct translations in the target language. This is necessary for machine translation systems such as Google Translate.
5. **Question answering:** Parsing is used in question-answering systems to help break down a question into its grammatical components, allowing the system to search a corpus for relevant replies.
6. **Text summarization:** Parsing is the process of extracting the essential syntactic and semantic structures of a text, which is necessary for producing short and coherent summaries.

Dynamic Programming Parsing

- To avoid extensive repeated work, must cache intermediate results, i.e. completed phrases.
- Caching (memoizing) critical to obtaining a polynomial time parsing (recognition) algorithm for CFGs.
- Dynamic programming algorithms based on both top-down and bottom-up search can achieve $O(n^3)$ recognition time where n is the length of the input string.

Dynamic Programming Parsing Methods

- CKY (Cocke-Kasami-Younger) algorithm: bottom-up, requires normalizing the grammar.
- Earley Parser - top-down, does not require normalizing grammar, more complex.

More generally, chart parsers retain completed phrases in a chart and can combine top-down and bottom-up searches.

Cocke–Younger–Kasami (CYK) Algorithm

Grammar denotes the syntactical rules for conversation in natural language. But in the theory of formal language, grammar is defined as a set of rules that can generate strings. The set of all strings that can be generated from a grammar is called the language of the grammar.

Context Free Grammar: We are given a Context Free Grammar G = (V, X, R, S) and a string w, where:

- V is a finite set of variables or non-terminal symbols,
- X is a finite set of terminal symbols,
- R is a finite set of rules,
- S is the start symbol, a distinct element of V, and
- V and X are assumed to be disjoint sets.

The Membership problem is defined as: Grammar G generates a language L(G). Is the given string a member of L(G)?

Chomsky Normal Form: A Context Free Grammar G is in it if each rule if each rule of G is of the form:

- A --> BC, [with at most two non-terminal symbols on the RHS]
- A --> a, or [one terminal symbol on the RHS]
- S --> nullstring, [null string]

Cocke-Younger-Kasami Algorithm

It is used to solve the membership problem using a dynamic programming approach. The algorithm is based on the principle that the solution to problem [i, j] can be constructed from solution to subproblem [i, k] and solution to sub problem [k, j]. The algorithm requires the Grammar G to be in Chomsky Normal Form (CNF). Note that any Context-Free Grammar can be systematically converted to CNF. This restriction is employed so that each problem can only be divided into two subproblems and not more - to bound the time complexity.

How does the CYK Algorithm work?

For a string of length N, construct a table T of size N x N. Each cell in the table T[i, j] is the set of all constituents that can produce the substring spanning from position i to j. The process involves filling the table with the solutions to the subproblems encountered in the bottom-up parsing process. Therefore, cells will be filled from left to right and bottom to top.

	1	2	3	4	5
1	[1, 1]	[1, 2]	[1, 3]	[1, 4]	[1, 5]
2		[2, 2]	[2, 3]	[2, 4]	[2, 5]
3			[3, 3]	[3, 4]	[3, 5]
4				[4, 4]	[4, 5]
5					[5, 5]

In T[i, j], the row number i denotes the start index and the column number j denotes the end index.

- $A \in T[i,j]$ if and only if $B \in T[i,k], C \in T[k,j]$ and $A \rightarrow BC$ is a rule of G
- $A \in T[i,j]$ if and only if $B \in T[i,k], C \in T[k,j]$ and $A \rightarrow BC$ is a rule of G

The algorithm considers every possible subsequence of letters and adds K to T[i, j] if the sequence of letters starting from i to j can be generated from the non-terminal K. For subsequences of length 2 and greater, it considers every possible partition of the subsequence into two parts, and checks if there is a rule of the form A ? BC in the grammar where B and C can generate the two parts respectively, based on already existing entries in T. The sentence can be produced by the grammar only if the entire string is matched by the start symbol, i.e, if S is a member of T[1, n].

Consider a sample grammar in Chomsky Normal Form:

- NP --> Det | Nom
- Nom --> AP | Nom
- AP --> Adv | A
- Det --> a | an
- Adv --> very | extremely
- AP --> heavy | orange | tall
- A --> heavy | orange | tall | muscular
- Nom --> book | orange | man

Now consider the phrase, "a very heavy orange book":

a(1) very(2) heavy (3) orange(4) book(5)

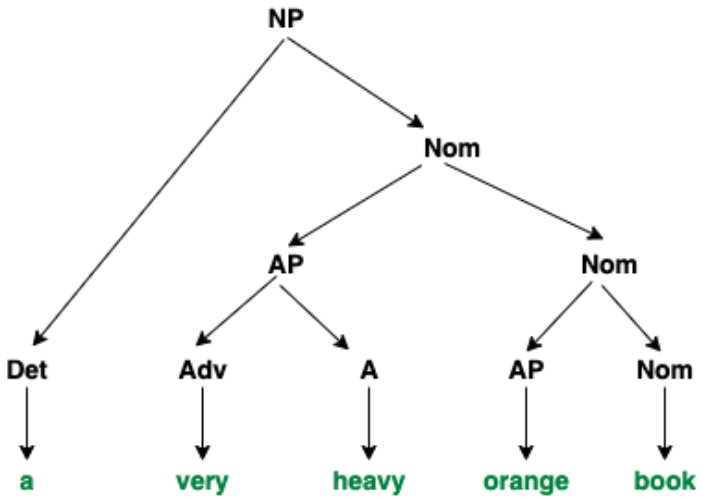
Let us start filling up the table from left to right and bottom to top, according to the rules described above:

	1 a	2 very	3 heavy	4 orange	5 book
1 a	Det	-	-	NP	NP
2 very		Adv	AP	Nom	Nom
3 heavy			A, AP	Nom	Nom
4 orange				Nom, A, AP	Nom
5 book					Nom

The table is filled in the following manner:

1. T[1, 1] = {Det} as Det --> a is one of the rules of the grammar.
2. T[2, 2] = {Adv} as Adv --> very is one of the rules of the grammar.
3. T[1, 2] = {} as no matching rule is observed.
4. T[3, 3] = {A, AP} as A --> very and AP --> very are rules of the grammar.
5. T[2, 3] = {AP} as AP --> Adv (T[2, 2]) A (T[3, 3]) is a rule of the grammar.
6. T[1, 3] = {} as no matching rule is observed.
7. T[4, 4] = {Nom, A, AP} as Nom --> orange and A --> orange and AP --> orange are rules of the grammar.
8. T[3, 4] = {Nom} as Nom --> AP (T[3, 3]) Nom (T[3, 4]) is a rule of the grammar.
9. T[2, 4] = {Nom} as Nom --> AP (T[2, 3]) Nom (T[4, 4]) is a rule of the grammar.
10. T[1, 4] = {NP} as NP --> Det (T[1, 1]) Nom (T[2, 4]) is a rule of the grammar.
11. T[5, 5] = {Nom} as Nom --> book is a rule of the grammar.
12. T[4, 5] = {Nom} as Nom --> AP (T[4, 4]) Nom (T[5, 5]) is a rule of the grammar.
13. T[3, 5] = {Nom} as Nom --> AP (T[3, 3]) Nom (T[4, 5]) is a rule of the grammar.
14. T[2, 5] = {Nom} as Nom --> AP (T[2, 3]) Nom (T[4, 5]) is a rule of the grammar.
15. T[1, 5] = {NP} as NP --> Det (T[1, 1]) Nom (T[2, 5]) is a rule of the grammar.

We see that T[1][5] has NP, the start symbol, which means that this phrase is a member of the language of the grammar G. The parse tree of this phrase would look like this:



Robust and Scalable Parsing on Noisy Text as in Web documents

- Noisy text refers to input data that contains errors, informal language, spelling mistakes, non-standard grammar, abbreviations, and artifacts like HTML tags, emojis, and inconsistent formatting.
- Web documents (social media, forums, OCR scans, crowdsourced data) are major sources of noisy text, complicating NLP parsing tasks.

Parsing Challenges in Noisy Text

- Out-of-Vocabulary Words: Unusual terms, slang, domain-specific vocabulary, or misspellings that standard parsers cannot recognize.
- Grammatical Inconsistency: Unstructured, "bullet-point" writing; lack of proper sentence boundaries; fragments; dropped subjects or verbs.
- Orthographic Issues: Mixed cases, non-ASCII symbols, emoticons, HTML artifacts embedded in the text.
- Ambiguity: Syntactic ambiguity increases with irregular phrasing and incomplete structures; more dependency on contextual or probabilistic models.

Preprocessing and Normalization

Techniques to reduce text noise before parsing:

- Tokenization: Splitting text into tokens or words—a challenge when text lacks clear boundaries.
- Lowercasing and Unicode Normalization: Standardizing all text to lowercase and a uniform Unicode format.
- Spell Checking and Correction: Using dictionaries and context-sensitive algorithms to correct spelling.
- Removal of Non-Linguistic Artifacts: Stripping HTML/XML tags, scripts, and metadata not needed by the parser.
- Text Normalization: Expanding contractions, standardizing abbreviations, and mapping variants (u → you).
- Stemming/Lemmatization: Reducing words to their grammatical root or lemma for consistency.
- Stopword Removal: Eliminating very common words ("the", "a") to prevent them from affecting syntactic analysis.

Approaches to Robust Parsing

- Robust Statistical Parsers:
  - Train machine learning or statistical models on data that includes noisy or web-like samples to make them more error-tolerant.
  - Use probabilistic context-free grammars (PCFGs), dependency parsers, or neural network (LSTM/Transformer) models that can generalize from noisy training data.
- Hybrid Rule-Based and Probabilistic Models:
  - Combine hand-crafted rules (for easily modelled structure) and statistical models (for ambiguity and unseen cases) for improved robustness and scalability.
- Self-Tuning and Adaptation:
  - Adapt parsers by retraining or fine-tuning on the specific, noisy dataset at hand, rather than using generic models.
  - Use domain adaptation techniques and crowd-annotated corpora for web or domain-specific language.
- Error Correction in Pipeline:
  - Integrate spell-checkers, noise detectors, and normalizers directly into parsing pipelines to "clean" as parsing progresses.

Scalability Strategies

- Parallelization: Batch and distribute parsing tasks for large-scale document sets, leveraging cloud or multi-core architectures.
- Lightweight Models: Use simpler or approximate parsing methods where perfect accuracy is less important—for tasks like information retrieval or pre-filtering.
- Incremental Parsing: Parse new or changed portions of text rather than rerunning parsing from scratch after minor edits.

Key Algorithms/Frameworks

- CKY Algorithm: A classic dynamic-programming parser amenable to probabilistic grammar extensions for handling ambiguity.
- Dependency Parsers: Focus on local syntactic dependencies and often perform better on non-standard (noisy) text compared to phrase-structure parsers.
- Neural Approaches: Transformers and encoder-decoder architectures can be robust to noise if exposed during training; self-attention helps model context even in corrupted input.

Best Practices

- Tune and Validate Parsers on Noisy Validation Sets: Always evaluate with realistic, noisy corpora (not just clean, canonical datasets).
- Iterative Cleaning and Re-parsing: Employ multiple passes—clean, parse, re-clean, re-parse—to incrementally improve the final structure.
- Domain-Specific Spell Checks and Normalization: Customize pre-processing pipelines using dictionaries or language models tailored to the specific web domain.

Hybrid of Rule Based and Probabilistic Parsing:

Rule-Based Parsing relies on expert-defined linguistic rules (syntax, grammar, patterns) to parse sentences, ensuring precise handling of domain-specific language, edge cases, and deterministic constructs.

Probabilistic Parsing uses statistical models (like PCFGs, neural networks) to determine the most probable parse among possible trees using frequencies derived from annotated or untagged corpora, providing robustness to ambiguity and variability in natural language.

Hybrid Parsing combines these: rules guide deterministic segments, while probabilities tackle ambiguity, unstructured, or novel inputs. Probabilistic scores can be "bootstrapped" from rule-based outputs, helping the system favor parses that fit empirical usage patterns, without requiring extensive manual annotation.

Working Mechanism

- Deterministic Handling: Rules are first applied to analyze clear, domain-specific segments (e.g., known entity formats, rigid syntactic patterns).
- Statistical Guidance: Where multiple parses or ambiguities occur, the parser uses computed probabilities—based on corpus frequencies or statistical models—to prioritize the most likely interpretation.
- Bootstrapping: Some methods parse large, unannotated corpora with rules; the observed frequencies of parsing patterns are used to assign probabilities, which then guide future parses, giving a learned bias to the rule-based system.

Advantages

- Improved Coverage and Flexibility: Hybrid parsers handle both well-defined and ambiguous language phenomena, making them more adaptable to real-world language.
- Efficiency and Performance: Probabilities prune combinatorial parse possibilities, increasing speed; rules ensure high accuracy in critical domain segments.
- Domain Adaptability: Rules can be fine-tuned for specific tasks or languages, while probabilistic components generalize across domains using data.

Applications

- Grammar Checking: Hybrid systems detect idioms with statistical models and errors with deterministic rules, improving correction accuracy.
- Resume Parsing: Combining rule-based entity extraction (names, skills) with probabilistic segmentation for diverse CV formats increases parsing robustness.

- Low-Resource Language Processing: Hybrid parsing helps bootstrap effective systems when annotated data is scarce, especially vital in part-of-speech tagging and syntax parsing tasks.
- Medical/Niche Data: Rule-based modules cover strict terminology; probabilistic parsing identifies patterns for novel or ambiguous language in clinical notes.

Best Practices & Future Directions

- Use rule-based components for boundary cases and mission-critical segments, and probabilistic methods for ambiguity management and scaling to large, variable datasets.
- Continual refinement and integration ("bootstrapping") allow hybrid models to adapt over time without manual retraining or annotation.

Modern NLP research now often prefers hybrid solutions combining deep learning and symbolic/rule-based representations for more powerful language understanding and reasoning.

Scope ambiguity and attachment ambiguity: Scope ambiguity and attachment ambiguity are two of the most challenging problems in natural language processing (NLP), as they directly affect how sentences are parsed and interpreted. Resolving these ambiguities is essential for tasks like machine translation, question answering, and information extraction.

Scope Ambiguity in NLP

Scope ambiguity arises when it is unclear which part of a sentence governs the interpretation of operators or quantifiers—such as "every," "some," "a," or negations—leading to multiple possible meanings. This ambiguity is semantic in nature as it depends on how the meanings of components interact in the sentence.

- Example: "Every man loves a woman."
    - One interpretation (surface scope): For each man, there is possibly a different woman he loves.
    - Another interpretation (inverse scope): There is a particular woman that every man loves.
- Explanation: The ambiguity stems from whether the existential quantifier ("a woman") falls within the scope of "every man" or outside it. Different scope orderings produce different semantic interpretations.
- Importance: Resolving scope ambiguity often requires world knowledge or contextual information because both interpretations are semantically valid but pragmatically different.
- Formal semantics view: Scope ambiguity is seen as different ways of composing semantic operators, affecting the truth conditions of sentences.
- Challenges: In computational NLP systems, disambiguating between scope readings is difficult because it requires integrating syntax, semantics, and external knowledge.

Attachment Ambiguity in NLP

Attachment ambiguity, specifically prepositional phrase (PP) attachment ambiguity, occurs when it is unclear which part of the sentence a phrase (typically a prepositional phrase) modifies or "attaches" to.

- Example: "The man saw the girl with the telescope."
    - Did the man use the telescope to see the girl?
    - Or does the girl have the telescope?
- Explanation: The ambiguity lies in whether the prepositional phrase "with the telescope" modifies the verb phrase "saw" (instrumental) or the noun phrase "the girl" (descriptive).
- Significance: This type of ambiguity is syntactic and depends on parsing the sentence structure correctly.
- Additional cases: Attachment ambiguity also arises when sentences contain multiple prepositional phrases, increasing the potential attachment points and ambiguity.
- Resolution techniques: Statistical parsing, semantic distance measures, and machine learning methods that consider context and similarity with known examples are commonly used to resolve attachment ambiguity.

Techniques for Resolution

Scope Ambiguity:

- Use of semantic parsing frameworks to compute all possible scope orders.
- Integration of world knowledge or contextual cues to select the preferred reading.
- Machine learning models trained on annotated corpora with scope information, though labeled data is limited.
- Statistical and Machine Learning Models
    - Algorithms such as Naive Bayes, Logistic Regression, and Support Vector Machines have been used to statistically model scope disambiguation based on linguistic features extracted from annotated corpora.
    - Contemporary models like BERT, GPT, and other transformer-based architectures leverage large pretrained language models to predict the preferred scope reading by learning contextual cues from vast datasets.
    - Few-shot and zero-shot learning approaches are applied where models are prompted with example disambiguations to improve handling of unseen scope ambiguity cases.
- Logical Form Parsing
    - Generating logical forms or semantic representations that explicitly encode scope allows systems to compare alternative scope structures.
    - By computing possible interpretations as logical formulas, a ranking or preference model can pick the most plausible scope ordering based on context or domain knowledge.
- Contextual and Pragmatic Reasoning
    - Contextual information from surrounding sentences, speaker intent, and real-world knowledge is employed to prune unlikely scope interpretations.
    - Pragmatic models integrate world knowledge and typical discourse patterns to resolve scope.
    - Discourse analysis and focus of attention can influence scope resolution by emphasizing certain operators or quantifiers.

Attachment Ambiguity:

- Supervised learning approaches that use context features to predict the correct attachment.
- Use of semantic distance measures to gauge plausibility of attachment.
- Corpus-based methods, iterative clustering and similarity-based disambiguation.
- Leveraging prosodic or discourse-level cues (in spoken or larger text contexts).
- Corpus-Based and Example-Based Learning
    - Leveraging large annotated corpora where attachment decisions are labeled, machine learning classifiers can predict attachment based on features like word co-occurrence, verb-noun compatibility, and semantic similarity.
    - Example-based methods find the closest matching known attachment examples in training data and transfer the attachment decision.
- Semantic Similarity and WordNet-Based Measures
    - Calculating semantic distances between candidate attachment sites and the prepositional phrase using knowledge bases such as WordNet helps estimate plausibility of each attachment.
    - The nearest common ancestor in lexical hierarchies and path lengths between concepts determine semantic affinity guiding attachment.
- Syntactic Heuristics and Structural Preferences
    - Heuristics involving phrase length, attachment to the nearest noun phrase, or verb phrase influence attachment.
    - Some studies use relative clause length and the syntactic position of determiners or noun phrases to predict attachment likelihood, achieving results approximating human preferences.
- Prosodic and Multimodal Cues
    - In speech contexts, prosodic features (pitch accent, intonation) provide clues to attachment by highlighting certain words or phrases.

- Multimodal NLP integrates visual or acoustic information that can disambiguate references and attachments by linking text and context.

Comparison of Scope and Attachment Ambiguity

Aspect	Scope Ambiguity	Attachment Ambiguity
Nature	Semantic ambiguity involving scope of operators or quantifiers	Syntactic ambiguity involving phrase attachment
Common Ambiguous Items	Quantifiers (every, some, a) and negations	Prepositional phrases, clauses
Example Sentence	"Every man loves a woman"	"The man saw the girl with the telescope"
Source of Ambiguity	Multiple semantic interpretations from scope of operators	Multiple ways to attach phrases in parsing tree
Resolution Requirements	Semantic world knowledge and pragmatic context	Syntactic parsing and contextual/ semantic clues
NLP Techniques	Semantic parsing, world knowledge integration	Statistical parsing, machine learning, similarity measures